

Schema-Bound Tokens

Attestor Sets as Mint Authority on Attestation-Native Chains

Ligate Labs Research, Working Paper v0.1

Date: 2026-05-19

Contact: hello@ligate.io

Abstract

Most blockchains expose two token primitives at the runtime layer: fungible tokens (admin-mintable balances) and non-fungible tokens (admin-mintable unique items). The mint authority in both cases is a single address whose only on-chain accountability is its bonded stake, if any. A third primitive becomes natural on a chain that already runs attester sets as a first-class object: bind the mint authority to an `AttestorSetId` rather than to a single address, and make every mint event itself an attestation under a canonical system schema.

This note specifies that primitive (schema-bound tokens) as a research object. The engineering design lives in `ligate-chain#286`. The contribution here is the formal-properties analysis: authorization equivalence (any actor who can produce a valid attestation under the bound schema can authorize a mint, and conversely), auditability (mint events are queryable as attestations under `chain.token-mint/v1`, not as opaque state diffs), composition with the per-schema fee market, and liveness under attester-set turnover.

The differentiator from existing threshold-issuance patterns (multisig wallets, FROST-based protocols, EAS revocable attestations) is that Ligate’s threshold verification is *native to the chain’s attestation module*, not a separate contract or off-chain protocol. The reputation layer (Proof of Useful Attestation, v0.8) then provides an economic floor: bad-faith mints by the attester set damage the same reputation that backs every other attestation they sign. This is the on-thesis token primitive for an attestation-native chain. The chain’s distinguishing primitives carry the token’s authorization, audit trail, and economic security in one piece.

1. Background

1.1 The token-primitive landscape on Ligate Chain

Per `ligate-chain#286`, v1 of Ligate Chain exposes four token primitives at the runtime layer:

1. **Standard fungible tokens** (`ligate-chain#47`). Mint authority: single address. Issuance auditability: state-diff only. Familiar ERC-20 shape.
2. **Standard non-fungible tokens** (`ligate-chain#48`). Mint authority: single address. Issuance auditability: state-diff only. Familiar ERC-721 shape.
3. **Schema-bound fungible tokens**. Mint authority: `AttestorSetId` (threshold quorum). Mint events: attestations under `chain.token-mint/v1`.
4. **Schema-bound non-fungible tokens**. Same authorization model as (3), distinct asset shape.

The first two primitives are commodity. They exist so partners building on Ligate can deploy familiar token shapes without learning a new model. The last two are the architecturally on-thesis primitives. They use the chain’s distinguishing objects (schemas + attester sets) as the foundation of the token’s authorization model.

1.2 Why threshold mint authority is the right default

A single-address mint authority is a single point of compromise. Industry workarounds include multisig wallets and external threshold-signature protocols, but those are *bolted on* to chains that did not design for them. The chain sees the multisig contract’s output, not the quorum it represents. The auditability of “who actually authorized this mint” lives outside the chain.

A chain whose runtime already runs attester sets as first-class state objects can do better. The attester set’s threshold quorum is verified natively in the consensus pipeline. The mint event itself becomes an attestation, queryable under a system schema, with the same threshold-signature semantics as every other attestation on the chain. There is no separate ledger and no separate trust layer. The issuance log is part of the attestation log.

1.3 Position relative to PoUA

PoUA v0.8 establishes attester sets as the primary trust primitive for application-layer correctness, with non-transferable reputation tied to the validators who include valid attestations. Schema-bound tokens reuse the same attester-set object and inherit the same reputation feedback loop: if an attester set authorizes a fraudulent mint, the validators who include that attestation are subject to the same reputation mechanics that govern any §A.3 grinding behavior. The reputation layer is not retrofitted to handle tokens; tokens are simply another application of the layered defense PoUA already specifies.

2. The schema-bound primitive: formal definition

2.1 Mint authority binding

Each schema-bound token type is uniquely identified by

```
token_id = SHA-256(domain_tag || mint_authority || name || decimals)
```

where `mint_authority = AttesterSet(A)` for some registered attester set `A`. The token id is bound to the specific attester set at construction time. Subsequent rotation of the attester set (per Ligate Chain’s attester-set management; #286 §3.4) does not invalidate the token id.

2.2 Mint as attestation

A mint event that creates `n` units of token type `T` at recipient `r` is recorded as an attestation under the system schema `chain.token-mint/v1` (call it `sigma_mint`), with payload

```
p = (token_id(T), r, n, nonce, metadata)
```

signed by a threshold quorum of the attester set `A` at the threshold `k` already on record for `A`. The attestation is *valid* if and only if it carries a `k`-of-`|A|` threshold signature over `(p, sigma_mint, submitter)`.

2.3 Recall semantics

An attester set may optionally bind a token type with a `recall_by_authority` flag at construction. When set, the same attester set that authorized minting may also authorize burns from arbitrary holder balances, with the same threshold semantics. Recall is itself an attestation under `chain.token-burn/v1`; the audit trail is symmetric.

2.4 Recoverability under attester-set turnover

The token id binds to the attester set’s *registered identity* (`AttesterSetId`), not to its key material. When the attester set rotates keys (replaces members or threshold), existing minted balances remain

valid (mint events are historical attestations; their threshold-signature verification uses the keys recorded at the time of mint). Future mints use the rotated key set. This decouples token-balance persistence from key-rotation events, the same way the chain decouples attestation persistence from attessor-set membership changes.

3. Formal properties

This is the priority section: what’s actually provable about the primitive once §2 is fixed. Five formal claims, each stated and argued.

3.1 Authorization equivalence

Claim. Any actor that can produce a valid attestation under a schema σ whose attessor set is $\mathcal{A}$ can authorize a mint of any token type $\mathcal{T}$ whose `mint_authority` is `AttestorSet(A)`, and conversely.

Argument. Validity of both the schema attestation and the mint attestation reduces to the same predicate: a $k_{\mathcal{A}}\text{-of-}|\mathcal{A}|$ threshold signature over the canonical Borsh encoding of the payload (with the schema id as part of the signed bytes). The schema id differs (the application schema for one, σ_{mint} for the other), but the signing quorum is identical. Forward direction: a quorum that can produce a valid signature for schema σ can also produce a valid signature for σ_{mint} (the cryptographic primitive is the same). Reverse direction: a quorum that can produce a valid signature for σ_{mint} can also produce a valid signature for σ (same reasoning). The equivalence is symmetric. $\square$

Consequence. Designers who decide to trust an attessor set with schema attestation work have also implicitly decided to trust them with mint authorization for any token bound to that same set. This is a feature, not a bug: it forces the same trust-modeling discipline at one decision point rather than two.

3.2 Auditability via the attestation log

Claim. For any token type $\mathcal{T}$ schema-bound to $\mathcal{A}$, the full set of mint events that contributed to any holder’s balance is queryable as a finite set of attestations under σ_{mint} filtered by their `token_id` field matching `token_id(T)`.

Argument. Every mint is recorded as an attestation per §2.2. Attestations are stored in the chain’s attestation log (per PoUA v0.8 §3.7 system diagram). The token’s runtime state (current balances) is derivable from the cumulative sum of mint events minus burn events; both classes are attestations. There is no off-attestation-log state mutation for schema-bound tokens. $\square$

Consequence. Audit infrastructure that already indexes attestations (chain explorers, partner indexers, downstream analytics) gets token-issuance audit for free. No separate issuance ledger needs to be tracked. Investigators reconstructing “who minted what when” use the same queries as investigators reconstructing “who attested what when.”

3.3 Composition with the per-schema fee market

Claim. A mint event under σ_{mint} pays exactly the per-attestation fee for σ_{mint} , plus the standard chain gas for the transaction carrying it. The mint does not pay an additional, separate “token-mint

fee” on top of the attestation fee.

Argument and open question. The attestation module’s per-schema fee market (per the Per-Schema Fees paper, v0.1.1) prices attestations by schema. σ_{mint} is a system schema; its fee is set by governance and not subject to the open-fee-market dynamics that apply to application schemas. The bound mint fee should be calibrated so that:

- Routine mint volume does not exhaust block space (a fee floor)
- Adversarial mint flooding (a quorum that controls $\mathcal{A}$ tries to spam mints to inflate $\mathcal{T}$ ’s supply) is economically deterred (a fee ceiling-equivalent)

Open question. Whether the mint fee should also include a per-unit-minted component (a Pigouvian tax on supply expansion, recoverable to the chain’s τ_{burn} pool) is an economics RFC question, tracked at [ligate-chain#258](#). Without it, the per-attestation fee is the only cost regardless of n , which may be too cheap for adversarial high-supply mints. Documented here; not resolved at v0.1.

3.4 Liveness under attestor-set turnover

Claim. Holders of $\mathcal{T}$ retain valid balances when the attestor set $\mathcal{A}$ rotates members or threshold, provided the rotation is performed through the chain’s attestor-set management module (not via off-chain key compromise + re-registration).

Argument. Mint events are historical attestations. Their validity at the time of mint is determined by the threshold-signature verification under the keys recorded for $\mathcal{A}$ *at the slot of the mint attestation*. A rotation that changes $\mathcal{A}$ ’s key set at slot t updates the verifier state for attestations at slot $t' > t$ but does not retroactively invalidate attestations at slot $t' \leq t$. The historical attestation log is append-only by construction (per PoUA v0.8 §3.7); rotation events are recorded as their own attestations under the attestor-set management schema. Token balances derived from pre-rotation mints remain in the chain’s runtime state regardless of post-rotation key set. $\square$

Edge case. If $\mathcal{A}$ is *removed* entirely (governance action, end-of-life), the token type’s `mint_authority` is permanently revoked; the runtime can no longer accept new mints (no valid signing quorum exists), but pre-removal balances remain valid. This is the intended end-of-life path for a sunset token, similar to a smart-contract owner renouncing admin in conventional patterns.

3.5 Reputation feedback loop

Claim. Bad-faith mints by $\mathcal{A}$ are subject to the same reputation mechanics that govern any §A.3-grinding behavior under PoUA. There is no separate “token issuance reputation” track.

Argument. Mint events are attestations under σ_{mint} , included by validators in blocks. Per PoUA v0.8 §4.3 the reputation update rewards validators who include valid attestations and exposes them to the standard slashing conditions if those attestations are invalid or detected as part of a grinding pattern (§A.2 / §A.3). If $\mathcal{A}$ issues a fraudulent mint (a mint exceeding a stated cap, or a mint contradicting an off-chain authoritative source), the chain has the same recourse it has for any fraudulent attestation: detect, slash the validator who included it, and trigger appeal via §5.5.5.

Limit of this argument. The chain detects *invalid* attestations (failed threshold signature) and *graph-shaped misbehavior* (§A.3 bipartite-density). It does not detect *semantically incorrect* mints (the attestor set issues a mint that satisfies cryptographic validity but is contractually unauthorized). Semantic correctness is the schema designer’s problem, the same way it is for any application-layer

schema. The reputation feedback loop bounds the cost of provably-bad behavior, not the cost of debatable behavior.

4. Game-theoretic concerns (open questions)

Items the v0.1 draft surfaces explicitly. Each is potentially v0.2 or v1.0 work; not all need v0.1 answers.

1. **Attestor-set incentive to issue beyond stated cap.** If $\mathcal{A}$ stakes reputation through PoUA and that reputation has forward-revenue value (§6.3 of PoUA v0.8), does the marginal-revenue gain from issuing one excess unit of $\mathcal{T}$ exceed the marginal-reputation loss? The answer depends on token-economic parameters (price of $\mathcal{T}$, depth of the market, expected detection probability) that are not yet specified. Documented as a v0.2 mechanism-design follow-up.
2. **Reputation impact of “cap exceedance” should be slashable.** A cap-exceedance slash trigger (parameterized by per-token-type max-supply) is the cleanest mechanism to make the reputation feedback loop binding for the supply dimension. The attestation module’s slashing module (ligate-chain#51) would need a new severity class for “schema-bound-token cap exceedance.” Not resolved at v0.1.
3. **Recall as governance lever vs user-protection backstop.** The `recall_by_authority` flag is a powerful primitive. Designers should be able to use it for compliance recall (e.g., regulated stablecoin), but holders should have some notice or appeal window. A 7-day notice period before recall executes (mirroring the §5.5.5 appeal window for PoUA slashes) is a natural default. Documented; concrete parameter calibration is v0.2.
4. **Sub-quorum partial mint.** Whether k -of- $|\mathcal{A}|$ should be the same threshold for mint authorization as for schema attestation, or whether mint authorization should require a *stricter* threshold (e.g., $k+1$ -of- $|\mathcal{A}|$), is an open design choice. v0.1 specifies the same threshold (§2.2) for simplicity; a stricter mint threshold would be a useful extension for tokens with higher economic stakes.
5. **Sublicensing via meta-schemas.** Can an attestor set delegate mint authority for a derived token type to a smaller sub-quorum without rotating the parent set? Open. Closest existing primitive is the cross-schema composition typed reference (Cross-Schema Composition paper); whether the composition primitive extends naturally to mint-authority delegation is a v0.2+ question.

5. Comparison to existing patterns

Pattern	Mint authority	Issuance auditability	On-chain quorum verification	Reputation tie-in
Standard ERC-20 (admin role)	Single address	State diff only	None	None

Pattern	Mint authority	Issuance auditability	On-chain quorum verification	Reputation tie-in
Safe / multisig wallet ERC-20	M-of-N approval contract	State diff only	Contract executes; chain sees the result	None
Threshold-signature issuance (e.g., FROST-based protocols)	Threshold of distributed keys	Signed transactions on the issuance chain	Yes, but bolted on per protocol	Varies by protocol
EAS revocable attestations	Schema resolver contract	On-chain but each attestation is its own contract call	Resolver-dependent	None
Schema-bound token (Ligate)	AttestorSetId	Mint events ARE attestations under <code>chain.token-mint/v1</code>	Native chain primitive (consensus pipeline)	PoUA reputation flows to the attestor set + including validators

The differentiators worth naming:

- **Threshold verification is in the consensus pipeline**, not in a contract. The chain natively understands what an attestor set is, what its threshold is, and how to verify a quorum signature. There is no separate primitive to deploy and audit.
- **Mint events are queryable as attestations** under a system schema. There is no separate token-issuance ledger; the audit infrastructure that indexes attestations is the audit infrastructure for token issuance.
- **Reputation flows through** to the same attestor set that authorized the mint. Fraudulent mints face the same economic floor (Lemma 1 in PoUA §5.5.3) as any other grinding behavior.

Neither Safe-style multisig nor EAS achieves any of these in a single integrated package. They each solve a slice. Schema-bound tokens are the integrated primitive.

6. Concrete use cases

Four use cases per ligate-chain#286. One worked through here in detail (use case B); three sketched.

6.1 (A) Regulated digital currency

Sketch only. A central bank or consortium issues a stablecoin whose mint authority is an attestor set of the consortium's members. Each mint event is queryable under `chain.token-mint/v1`. Regulators audit by querying the attestation log under that schema filtered by `token_id`. Recall is enabled to satisfy regulatory burn-on-court-order requirements. v0.2 should specify the recall notice period and the governance interaction surface.

6.2 (B) AI-provenance content as NFTs (the worked use case)

Themisra v1 ships canonical AI-provenance schemas: `themisra.proof-of-prompt/v1`, `themisra.content-provenance/v1`. A natural extension is to allow content authors to mint a *unique, verifiable token* representing the AI-provenance receipt for a specific artifact. The mint authority is the same attester set that signs the underlying provenance attestation.

Construction. When a content author submits a piece of AI-generated content for attestation, the Themisra attester set signs a `themisra.content-provenance/v1` attestation. The author then mints an NFT under a schema-bound token type whose mint authority is *the same attester set*. The NFT’s payload includes the hash of the original content + the attestation id of the provenance record. The token is a 1-of-1 mint (NFT semantics) with the content hash as the implicit uniqueness key.

Why this is interesting. Compared to the existing pattern of “issue an NFT on an EVM chain referencing an off-chain attestation,” the schema-bound construction has three advantages:

1. The NFT cannot exist without the provenance attestation existing first. The same attester set authorizes both. There is no race condition where the NFT mints before the attestation, or persists after the attestation is invalidated.
2. The audit trail for the NFT and the audit trail for the provenance record are *the same audit trail*. A buyer verifying the NFT’s authenticity verifies the provenance in one query.
3. If the provenance attestation is later shown to be fraudulent (e.g., the content was actually human-produced but mis-attested as AI), the recall mechanism can burn the NFT, and the §A.3 reputation feedback loop slashes the attester set’s reputation. The recall is a feature of the integrated trust model, not a hack bolted on.

v0.2 work. Specify the on-chain reference from the NFT to the provenance attestation (likely a `provenance_attestation_id: AttestationId` field in the NFT’s payload), the recall conditions (when can the attester set burn? automated trigger if the underlying provenance is invalidated, or only via governance?), and the marketplace-side queries (how does an NFT marketplace check that the linked provenance is still valid?).

Connection to chain#384 (Themisra Prompt Marketplace). The prompt-marketplace work uses similar machinery: an attester set authorizes prompt-template publication, the template is sold with royalty terms, and each downstream use is itself an attestation. The schema-bound NFT primitive is the natural data type for the templates themselves; the marketplace then routes royalties via the per-schema fee mechanism documented in the Per-Schema Fees paper.

6.3 (C) DAO treasury tokens

Sketch only. A DAO issues governance tokens whose mint authority is an attester set representing the DAO’s elected representatives. Issuance events are queryable. Recall is *not* enabled (DAO members hold balances permanently). Threshold rotation matches the DAO’s election cycle.

6.4 (D) Regulated licenses as NFTs

Sketch only. A regulator (e.g., a professional licensing board) issues licenses as NFTs whose mint authority is the regulator’s attester set. Issuance is auditable. Recall is enabled and routes through the regulator’s existing revocation process. Each license NFT carries the licensee’s identifier and the license terms in its payload.

7. Where this lives in the canon

Per [ligate-research#84](#) §7, this note is a **standalone research note** in [papers/schema-bound-tokens/](#). It is not absorbed into PoUA (which would expand PoUA’s scope beyond consensus weighting) and not folded into Native Delegation (which has a different thematic focus on validator-attestor key separation).

The note cross-links to:

- PoUA v0.8 for the attestor-set + threshold-signature mechanics that this note builds on
 - Per-Schema Fees v0.1.1 for the fee-market integration in §3.3
 - Cross-Schema Composition v0.1.1 for the typed-reference primitive that may extend to sub-licensing (§4.5)
 - [ligate-chain#286](#) for the engineering RFC
 - [ligate-chain#258](#) for the \$LGT economics
 - [ligate-chain#47](#) and [#48](#) for the standard token primitives
-

Roadmap

- **v0.1 (this draft, 2026-05-19)**: formal properties §3 written; one use case (B, AI-provenance NFTs) worked through; comparison table populated; game-theoretic open questions listed.
 - **v0.2 (target Q3 2026, post-devnet)**: resolve open questions §4.1 and §4.3 (attestor-set cap exceedance incentive analysis; recall notice-period calibration). Add §3.6 formal property on fee-market composition once Per-Schema Fees hits v0.2 mechanism. Add use case A (regulated currency) full worked example.
 - **v1.0 (post-mainnet)**: stable. Either folded into a “Token Primitives on Ligate” survey paper alongside [#47](#), [#48](#), or kept as a standalone reference for this specific primitive.
-

Out of scope for this note

- Engineering design (lives in [ligate-chain#286](#))
 - \$LGT economics around mint fees (lives in [ligate-chain#258](#))
 - Token contract code (lives in [ligate-chain#47](#), [#48](#), follow-up implementation issues)
 - EVM-compatible ERC-20 wrapping (lives in [ligate-chain#52](#))
-

End of working paper v0.1. Comments welcome to hello@ligate.io.